



DATABASES — dis-bot v2.0 Stable

Документ фиксирует **схемы SQLite** и **ownership (владение) таблицами** в билде **dis-bot v2.0**.



В v2.0 схемы считаются **стабильными**. Любые новые таблицы — только как расширения, без ломки существующих.



Общие правила хранения данных

- Хранилище: **SQLite**
- Данные разделены по доменам в отдельных файлах БД:
- `database/social.db`
- `database/reminders.db`
- `database/birthdays.db`
- Таблицы создаются **лениво** (`CREATE TABLE IF NOT EXISTS`) в модулях, которые ими владеют.
- Запрещены кросс-базовые JOIN и foreign keys между разными БД.

`social.db` — экономика, социалка, игры, ачивки, статистика

Назначение: центральная БД активности.

Таблицы и владельцы

Таблица	Назначение	Owner (источник истины)
<code>coins_wallet</code>	Текущий баланс монет	<code>core/economy.py</code>
<code>coin_ledger</code>	Журнал транзакций монет	<code>core/economy.py</code>
<code>daily_rewards</code>	Последний дейлик и серия	<code>fun_slesh/daily.py</code>
<code>reputation</code>	Репутация (кто кому выдал, дата)	<code>fun_slesh/rep_and_mood.py</code>
<code>mood</code>	Настроение (оценка, дата)	<code>fun_slesh/rep_and_mood.py</code>
<code>achievements</code>	Выданные достижения	<code>fun_slesh/achievements_engine.py</code>
<code>rps_duels</code>	PvP-дуэли КНБ (статусы/ходы/таймаут)	<code>fun_slesh/games.py</code>

Таблица	Назначение	Owner (источник истины)
msg_stats_daily	Суточные агрегаты сообщений	fun_slesh/ message_and_voice_stats.py
msg_index_checkpoints	Чекпоинты индексации по каналам	fun_slesh/ message_and_voice_stats.py
voice_sessions	Сессии войса	fun_slesh/ message_and_voice_stats.py
voice_totals_daily	Суточные агрегаты войса	fun_slesh/ message_and_voice_stats.py

Схемы таблиц (DDL)

coins_wallet (owner: core/economy.py)

```
CREATE TABLE IF NOT EXISTS coins_wallet (
  user_id    INTEGER PRIMARY KEY,
  balance    INTEGER NOT NULL DEFAULT 0,
  updated_at TEXT NOT NULL
);
```

coin_ledger (owner: core/economy.py)

```
CREATE TABLE IF NOT EXISTS coin_ledger (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id     INTEGER NOT NULL,
  delta       INTEGER NOT NULL,
  reason      TEXT NOT NULL,
  meta        TEXT,
  created_at  TEXT NOT NULL
);
```

daily_rewards (owner: fun_slesh/daily.py)

```
CREATE TABLE IF NOT EXISTS daily_rewards (
  user_id      INTEGER PRIMARY KEY,
  last_claim_msk TEXT NOT NULL,  -- YYYY-MM-DD (MSK)
  streak       INTEGER NOT NULL DEFAULT 0
);
```

reputation (owner: fun_slesh/rep_and_mood.py)

```
CREATE TABLE IF NOT EXISTS reputation (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  user_id     INTEGER NOT NULL,  
  given_by    INTEGER NOT NULL,  
  date        TEXT NOT NULL  
);
```

mood (owner: fun_slesh/rep_and_mood.py)

```
CREATE TABLE IF NOT EXISTS mood (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  user_id     INTEGER NOT NULL,  
  mood        INTEGER NOT NULL,  
  date        TEXT NOT NULL  
);
```

achievements (owner: fun_slesh/achievements_engine.py)

```
CREATE TABLE IF NOT EXISTS achievements (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  user_id     INTEGER NOT NULL,  
  code        TEXT NOT NULL,  
  title       TEXT NOT NULL,  
  awarded_at  TEXT NOT NULL,  
  UNIQUE(user_id, code)  
);
```

rps_duels (owner: fun_slesh/games.py)

```
CREATE TABLE IF NOT EXISTS rps_duels (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  guild_id    INTEGER NOT NULL,  
  channel_id  INTEGER NOT NULL,  
  initiator_id INTEGER NOT NULL,  
  opponent_id INTEGER NOT NULL,  
  init_choice TEXT,  
  opp_choice  TEXT,  
  status      TEXT NOT NULL,      -- open|done|cancelled|expired  
  created_at  TEXT NOT NULL,      -- ISO UTC  
  expires_at  TEXT NOT NULL       -- ISO UTC  
);  
  
CREATE INDEX IF NOT EXISTS idx_rps_duels_status ON rps_duels(status);  
CREATE INDEX IF NOT EXISTS idx_rps_duels_channel ON rps_duels(channel_id);
```

msg_stats_daily (owner: fun_slesh/message_and_voice_stats.py)

```
CREATE TABLE IF NOT EXISTS msg_stats_daily (  
  user_id    INTEGER NOT NULL,  
  guild_id   INTEGER NOT NULL,  
  channel_id INTEGER NOT NULL,  
  date       TEXT     NOT NULL, -- YYYY-MM-DD (UTC)  
  messages   INTEGER NOT NULL DEFAULT 0,  
  words      INTEGER NOT NULL DEFAULT 0,  
  emojis     INTEGER NOT NULL DEFAULT 0,  
  chars      INTEGER NOT NULL DEFAULT 0,  
  PRIMARY KEY (user_id, guild_id, channel_id, date)  
);
```

msg_index_checkpoints (owner: fun_slesh/message_and_voice_stats.py)

```
CREATE TABLE IF NOT EXISTS msg_index_checkpoints (  
  channel_id    INTEGER PRIMARY KEY,  
  last_message_id INTEGER  
);
```

voice_sessions (owner: fun_slesh/message_and_voice_stats.py)

```
CREATE TABLE IF NOT EXISTS voice_sessions (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  user_id     INTEGER NOT NULL,  
  guild_id    INTEGER NOT NULL,  
  channel_id  INTEGER NOT NULL,  
  started_at  TEXT     NOT NULL, -- ISO UTC  
  ended_at    TEXT,          -- ISO UTC  
  seconds     INTEGER NOT NULL DEFAULT 0  
);
```

voice_totals_daily (owner: fun_slesh/message_and_voice_stats.py)

```
CREATE TABLE IF NOT EXISTS voice_totals_daily (  
  user_id  INTEGER NOT NULL,  
  guild_id INTEGER NOT NULL,  
  date     TEXT     NOT NULL, -- YYYY-MM-DD (UTC)  
  seconds  INTEGER NOT NULL DEFAULT 0,  
  PRIMARY KEY (user_id, guild_id, date)  
);
```



reminders.db — напоминания, таймеры, временные роли

Назначение: хранение задач, переживающих рестарт.

Таблицы и владельцы

Таблица	Назначение	Owner
reminders	Напоминания с доставкой в канал/ЛС	fun_slesh/tools.py
timers	Сохранённые счётчики событий	fun_slesh/tools.py
temp_roles	Временные роли (когда снять)	fun_slesh/mem_roles.py

Схемы (DDL)

reminders (owner: fun_slesh/tools.py)

```
CREATE TABLE IF NOT EXISTS reminders (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id     INTEGER NOT NULL,
  channel_id  INTEGER,
  text        TEXT NOT NULL,
  remind_at   TEXT NOT NULL
);
```

timers (owner: fun_slesh/tools.py)

```
CREATE TABLE IF NOT EXISTS timers (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id     INTEGER NOT NULL,
  event       TEXT NOT NULL,
  target_date TEXT NOT NULL
);
```

temp_roles (owner: fun_slesh/mem_roles.py)

```
CREATE TABLE IF NOT EXISTS temp_roles (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  guild_id    INTEGER NOT NULL,
  user_id     INTEGER NOT NULL,
  role_id     INTEGER NOT NULL,
  role_name   TEXT NOT NULL,
  remove_at_utc TEXT NOT NULL
);
```



birthdays.db — дни рождения

Назначение: минимальная БД только для ДР.

Таблицы и владельцы

Таблица	Назначение	Owner
birthdays	Дата дня рождения пользователя (ДД.ММ)	fun_slesh/birthday.py

Схема (DDL)

birthdays (owner: fun_slesh/birthday.py)

```
CREATE TABLE IF NOT EXISTS birthdays (
  user_id INTEGER PRIMARY KEY,
  birthday TEXT NOT NULL
);
```



Ownership rules

Что значит «owner»

Owner — модуль, который: - определяет схему таблицы; - отвечает за корректность записей; - вносит изменения строго в рамках своей ответственности.

Запрещено

- добавлять колонки в чужую таблицу без версии/миграций;
- писать в таблицу «чужого» домена;
- менять формат времени/ключей без договора (см. EVENTS.md).



Добавление новых таблиц после v2.0

Разрешено: - создавать новые таблицы в существующих БД, - или заводить новую БД под новый домен,

но: - без изменения/ломки существующих схем; - с явным описанием ownership.

DATABASES.md фиксирует схемы и владение таблицами dis-bot v2.0 Stable.